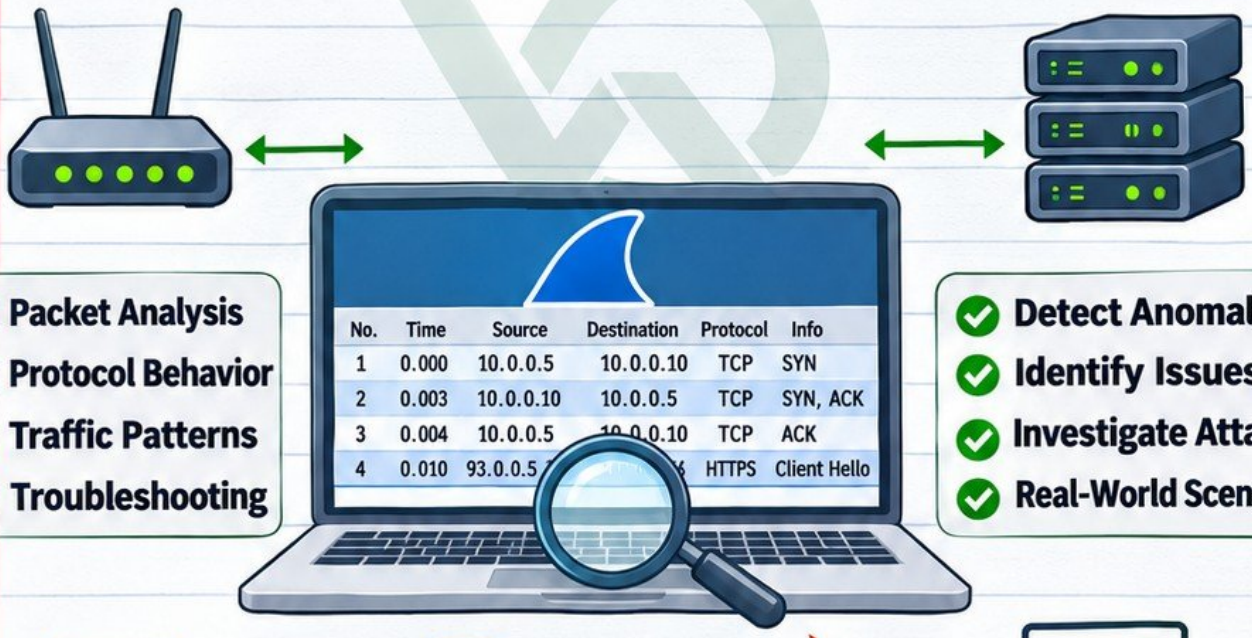
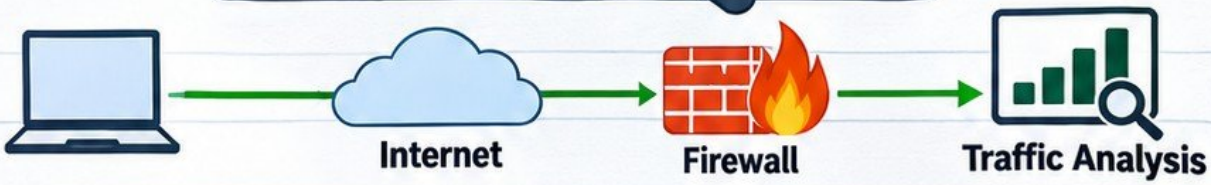
VERIQTA

How to Read Network Traffic Like a Detective



- Packet Analysis
- Protocol Behavior
- Traffic Patterns
- Troubleshooting

- Detect Anomalies
- Identify Issues
- Investigate Attacks
- Real-World Scenarios



A Practical Guide to Network Analysis, Troubleshooting and Security Investigation

@veriqta | @veriqta | @veriqta | @veriqta | @veriqta | @veriqta

VERIQTA

Every Network Request Leaves Evidence

Key Takeaways

- What network traffic actually is and why engineers inspect it.
- How applications exchange data through packets.
- Understand source, destination, protocol, and direction.
- Follow a browser request from a laptop to a web server.

What Is Network Traffic?

Network traffic is the data that moves between devices, applications and services over a network. It leaves a trace that can be captured, analyzed and used to find issues.

The Building Blocks of a Network Request



Source (who is sending)



Destination (who is receiving)



Protocol (how data is sent) e.g., HTTP, HTTPS, TCP



Direction (request and response)

A Browser Request in Action



1. Laptop (Client)



2. Network (Your Internet)



3. Internet (Route to Server)



4. Web Server (e.g., example.com)

1. You type a website (e.g., example.com) in your browser.
2. Your laptop sends a request (e.g., HTTP/HTTPS) to the server.
3. The request travels through your network and the internet.
4. The server processes it and sends a response (the web page).

Detective Insight



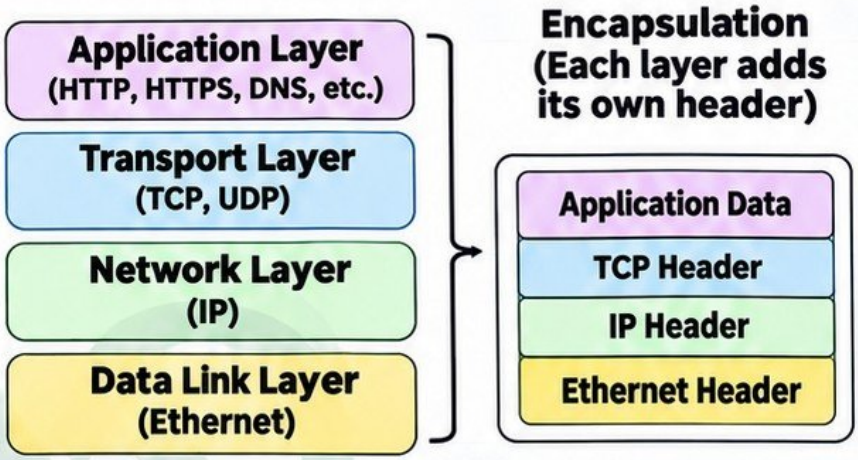
A failed website does not automatically mean the server is down.

Read a Packet Without Getting Lost

Key Takeaways

- Understand Ethernet frames, IP packets, TCP segments, and application data.
- Identify source and destination IP addresses, ports, and protocol fields.
- Understand encapsulation and how data is wrapped at each layer.
- Read a packet summary and determine who is communicating.

The Network Layers and Encapsulation



What Each Part Does

Layer / Part	What It Contains	Example
Ethernet Frame	Local network delivery (MAC addresses)	Src MAC: 00:1A:2B:3C:4D:5E
IP Packet	Network addressing (IP addresses)	Src IP: 192.168.1.10 Dest IP: 93.184.216.34
TCP Segment	Connection and data control (ports, flags)	Src Port: 54012 Dest Port: 443 Flag: SYN
Application Data	The actual request or response (e.g., HTTP)	GET / HTTP/1.1

Sample Packet Summary

No.	Time	Source	Destination	Protocol	Info
1	0.000	192.168.1.10	93.184.216.34	TCP	54012 → 443 [SYN]

- Source IP: 192.168.1.10 (your laptop)
- Destination IP: 93.184.216.34 (web server)
- Source Port: 54012 (ephemeral port from the client)
- Destination Port: 443 (HTTPS)
- Protocol: TCP (connection setup, SYN)

Practice



Determine which device initiated a connection from the sample packet.

Answer: 192.168.1.10 (the client) initiated the connection.

Follow the Five Clues of a Connection

Key Takeaways

- Understand the five-tuple: source IP, source port, destination IP, destination port, protocol.
- Distinguish client ephemeral ports from server listening ports.
- Learn why engineers group packets into conversations.
- Interpret a realistic client-to-server connection.

The Five-Tuple (Connection Identifier)

A network connection (or conversation) is uniquely identified by five values:

Source IP (who is sending)	Source Port (client ephemeral port)	Destination IP (who is receiving)	Destination Port (service port, e.g., 80/443)	Protocol (e.g., TCP, UDP)

Client Port vs Server Port

- Client (Ephemeral) Port: A temporary, high-numbered port used by your device (e.g., 49152–65535).
- Server (Listening) Port: A fixed port where a service waits for connections (e.g., 80 for HTTP, 443 for HTTPS).

How a Connection Looks



Client
192.168.1.10
Port: 54021
(ephemeral)

→

Five-Tuple for This Connection

Source IP: 192.168.1.10
Source Port: 54021
Destination IP: 93.184.216.34
Destination Port: 443
Protocol: TCP

→



Web Server
93.184.216.34
Port: 443
(HTTPS)

Why Group Packets into Conversations?

- Many packets make up a single request and response.
- It helps you see the complete flow (e.g., connection, data, and close).
- It makes troubleshooting faster and clearer (e.g., find errors, delays, or resets).

⚠ Beginner Mistake

✗ **Assuming the source port is always the service port.**

Reality: The source port is usually a random, high-numbered (ephemeral) port from the client, not 80 or 443.

Capture Your First Network Evidence

Key Takeaways

- Use Wireshark and tcpdump to capture and inspect network traffic.
- Identify the correct network interface (e.g., eth0, ens33, wlan0).
- Capture a small, controlled traffic sample.
- Understand the tcpdump command and its options.
- Identify TCP, UDP, and ICMP traffic.

The Two Main Tools

Wireshark

- Graphical tool (GUI).
- Easy to use and analyze.
- Great for detailed packet inspection.

tcpdump

- Command-line tool.
- Lightweight and fast.
- Ideal for servers and remote systems.

Find the Correct Network Interface

List available interfaces:

```
ip addr show
or
ifconfig -a
```

Capture Traffic with tcpdump

```
sudo tcpdump -i <interface>  -nn -c 20
```

Option	Meaning
-i <interface>	Selects the network interface to capture (e.g., eth0, ens33, wlan0).
-nn	Do not resolve IP addresses or port numbers to names (faster and cleaner).
-c 20	Capture only 20 packets, then stop.

Why Use Elevated Privileges?

Capturing network traffic accesses low-level network functions, which requires root (sudo) or equivalent privileges for security and system protection.

Practice: Identify Traffic Types

Protocol	What to Look For (Example)
TCP	TCP, flags (SYN, ACK), and a port (e.g., 443, 80).
UDP	UDP and a port (e.g., 53 for DNS).
ICMP	ICMP (e.g., echo request/reply for ping).

⚠ Security Reminder

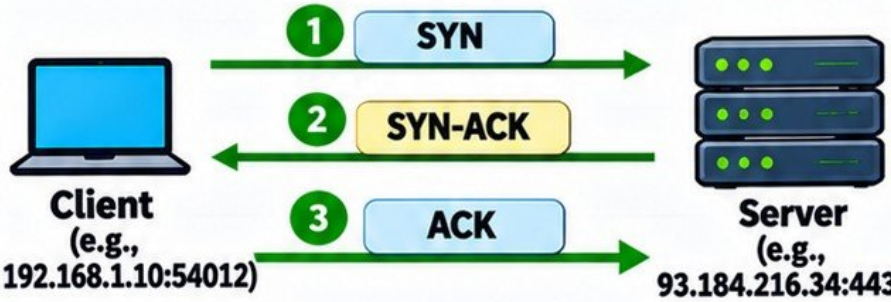
Capture only traffic that you are authorized to inspect. Do not collect or analyze unauthorized or sensitive traffic.

The TCP Handshake: Did the Connection Begin?

Key Takeaways

- Understand SYN, SYN-ACK, and ACK.
- Follow the three-way handshake packet by packet.
- Distinguish an established connection from an unanswered connection attempt.
- Explain why a successful handshake does not guarantee application health.
- Practice: Identify the client and server in a handshake.

The Three-Way Handshake



What Each Step Means

Step	Packet	Meaning
1	SYN	Client requests to start a connection (synchronize).
2	SYN-ACK	Server acknowledges and agrees to the connection.
3	ACK	Client confirms the connection.

Connection States

State	What It Means
Established	All three packets (SYN, SYN-ACK, ACK) are seen. The connection is open.
Unanswered Attempt	Only a SYN is seen and no SYN-ACK returns. The server may be down, blocked, or filtering the request.

Important Insight

A successful TCP handshake only proves that a network connection was established.
It does not mean the application (e.g., web, API) is working correctly.

Sample Handshake (Packet View)

No.	Time	Source → Destination	Protocol	Info
1	0.000	192.168.1.10 → 93.184.216.34	TCP	SYN
2	0.015	93.184.216.34 → 192.168.1.10	TCP	SYN, ACK
3	0.016	192.168.1.10 → 93.184.216.34	TCP	ACK

Practice

From the sample above:

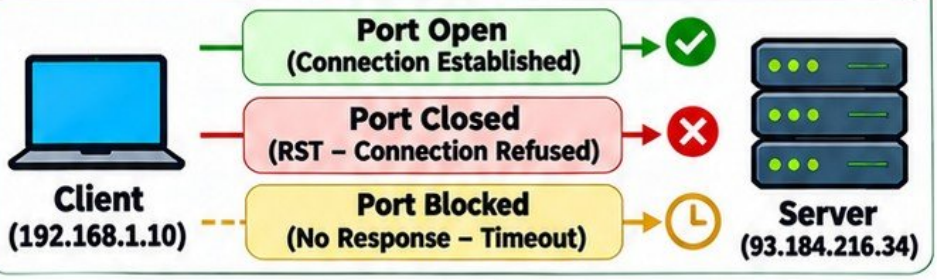
- Who is the client?
- Who is the server?
- Which packets show the handshake (SYN, SYN-ACK, ACK)?

When the Server Refuses to Talk

Key Takeaways

- Understand TCP RST, connection refused, and closed ports.
- Compare a refused connection with a timeout.
- Explain how a closed port differs from traffic silently dropped by a firewall.
- Use ss -lnt to check listening services.
- Identify whether the expected service is actually listening.

What Happens When You Connect?



RST, Refused, Timeout and Firewall

Event	Meaning
TCP RST	A reset packet that immediately ends the connection.
Connection Refused	The server is reachable but no service is listening on the target port (returns RST).
Timeout	No response from the server (often due to a firewall, packet drop, or network issue).
Closed Port	No application is listening on that port (typically returns RST).
Silently Dropped	Traffic is blocked without a response (common with firewalls).

Diagnostic Command

>_ **ss -lnt**

- ss: Display socket information.
- -l: Show only listening sockets.
- -n: Show IP addresses and ports (in numeric form, no name lookup).
- -t: Show TCP sockets.

Example: Check Listening Services

```
$ ss -lnt
State      Local Address:Port      Process
LISTEN     0.0.0.0:22              sshd
LISTEN     0.0.0.0:80              nginx
LISTEN     0.0.0.0:443             nginx
```

Quick Comparison

Scenario	What You See	Likely Cause
Port open	Connection succeeds	Service is listening
Connection refused	RST (immediate failure)	Port closed / no service
Timeout	No response (waits, then fails)	Firewall or network drop

Troubkshooting Steps

1. Run ss -lnt on the server.
2. Check if the expected port is listed.
3. If not listed, start or fix the service (e.g., nginx, httpd, or your app).
4. If listed but still unreachable, check the firewall and security group rules.

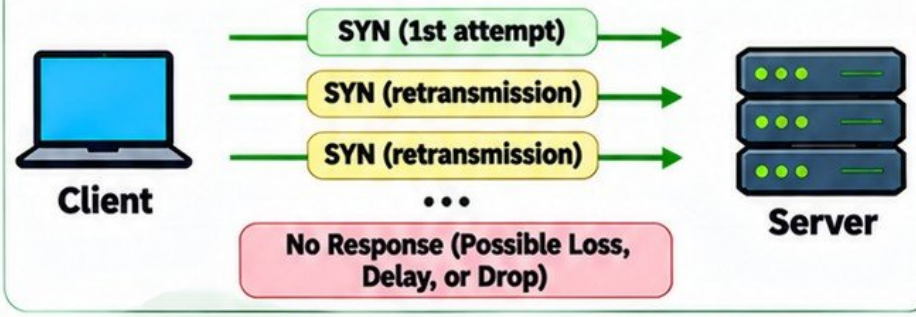
VERIQTA

The Mystery of the Missing Response

Key Takeaways

- Understand packet loss, retransmissions, and delayed responses.
- Recognize repeated SYN attempts and TCP retransmissions.
- Distinguish a missing response from a slow application.
- Know why retransmissions alone do not prove where packet loss occurred.
- Practice: Inspect a connection timeline and identify the first suspicious gap.

What Is Happening?



Important Terms

Term	Meaning
Packet Loss	A packet does not reach its destination.
Retransmission	A repeat of a packet because no response was received.
Delayed Response	A response arrives later than expected.
SYN Attempt	A request to start a TCP connection (handshake).

Retransmission Timeline (Example)

No.	Time (s)	Packet	Info
1	0.000	SYN	Client → Server (start connection)
2	1.000	SYN	Retransmission (no response)
3	3.000	SYN	Retransmission (no response)
4	7.000	SYN	Retransmission (still no response)

Missing Response vs Slow Application

Missing Response

- No reply to a packet (e.g., no SYN-ACK).
- Often caused by packet loss, firewall drop, or network issue.
- Triggers retransmissions.

Slow Application

- The server responds, but it takes longer than expected.
- Usually due to application processing, resource limits, or backend delays.

Why Retransmissions Are Not Proof

Retransmissions only show that a response was not received. They do not tell you whether the packet was lost on the client, the network, or on the server.

Practice

1. Review the timeline above.
2. Identify the first suspicious gap (where the expected response is missing).
3. State the most likely cause(s) and what you will check next.

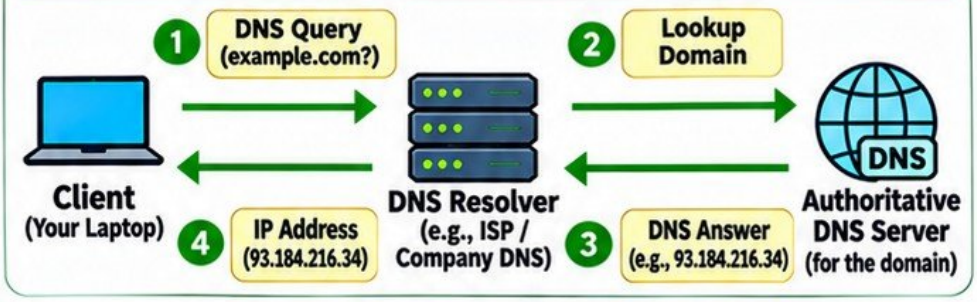
VERIQTA

DNS: When the Name Cannot Be Found

Key Takeaways

- Understand DNS queries, answers, resolvers, and caching.
- Follow a domain lookup from question to response.
- Interpret NOERROR, NXDOMAIN, and timeout behavior.
- Use dig to test and diagnose DNS resolution.
- Know that a DNS failure can stop the connection before any web request is sent.

How a DNS Lookup Works



Important DNS Responses

Result	Meaning	Example Use
NOERROR	The lookup was successful (the domain exists).	Returns an IP address (e.g., 93.184.216.34).
NXDOMAIN	The domain name does not exist (typo or not registered).	"examplee.com" → NXDOMAIN.
Timeout	No response from the DNS server (server unreachable or blocked).	Query times out (no answer).

Diagnostic Command

```
>_  dig example.com
```

- Sends a DNS query for example.com.
- Shows the answer, status, and TTL (cache time).
- Helps you confirm if the domain resolves and to which IP address.

Sample Output (Successful Lookup)

```
$ dig example.com
;; ANSWER SECTION:
example.com.    300    IN    A    93.184.216.34
;; Query time: 27 msec
;; SERVER: 8.8.8.8#53 (DNS resolver)
;; Status: NOERROR
```

DNS Failure vs Web Server Issue

Scenario	Where It Fails	What You See
DNS failure (NXDOMAIN)	Before any web connection	Domain not found (e.g., NXDOMAIN).
DNS timeout	DNS server does not respond	Lookup times out (no IP returned).
Web server issue	After DNS succeeds (IP is returned)	Connection fails or server returns an error (e.g., 5xx).

⚠ Common Failure

✕ **Blaming the web server when DNS resolution failed before any web connection began.**

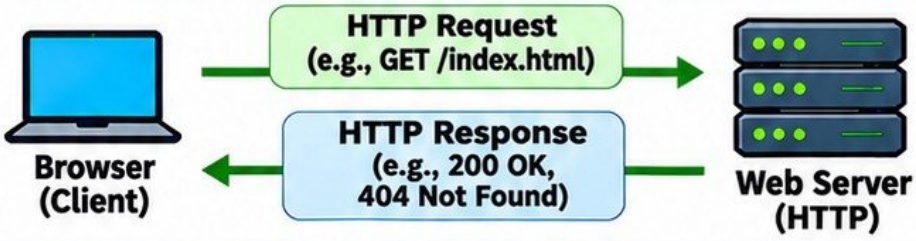
VERIQTA

HTTP: Read the Conversation Between Browser and Server

Key Takeaways

- Understand HTTP requests, responses, methods, headers, and status codes.
- Interpret GET requests and response status codes.
- Distinguish 200, 301, 404, 502, and 503.
- Use curl -I to inspect HTTP responses.
- Know that an HTTP error still means the application responded.

The HTTP Request–Response Flow



Common HTTP Methods

Method	Purpose (Example)
GET	Retrieve a resource (e.g., a web page).
POST	Send data to the server (e.g., a form).
PUT	Update or replace a resource.
DELETE	Remove a resource.

Key HTTP Status Codes

Code	Meaning	Typical Cause / Example
200	OK (Success)	The request succeeded (content returned).
301	Moved Permanently	The resource has a new permanent URL (redirect).
404	Not Found	The requested resource does not exist.
502	Bad Gateway	The server (often a proxy or load balancer) received an invalid response from an upstream server.
503	Service Unavailable	The server is temporarily unable to handle the request (e.g., maintenance or overload).

Diagnostic Command

```
>_ curl -I https://example.com
```

- Sends an HTTP HEAD request (headers only).
- Shows the response status code and key headers.
- Use it to quickly check if the site is reachable and what it returns.

Important Insight

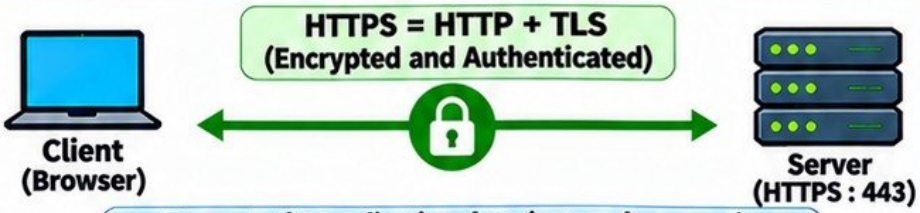
An HTTP error response (e.g., 404, 502, 503) still proves that the client reached the application and received a valid response.

HTTPS: Investigating Traffic You Cannot Read

Key Takeaways

- Understand Transport Layer Security (TLS).
- Explain why HTTPS encrypts application content.
- Identify observable connection metadata without decrypting traffic.
- Follow the simplified TLS handshake and certificate validation.
- Use openssl s_client to inspect an HTTPS connection.
- Know that encrypted traffic is normal and not automatically suspicious.

What HTTPS (TLS) Does



- Encrypts the application data (e.g., web content).
- Verifies the server's identity using a digital certificate.
- Protects the integrity of the data in transit.

Simplified TLS Handshake (Overview)

Step		What Happens
1	ClientHello	Client starts the TLS handshake (supported versions and ciphers).
2	ServerHello	Server selects the TLS version and cipher, and sends its certificate.
3	Certificate Check	Client validates the server's certificate (trusted and not expired).
4	Key Exchange	Client and server agree on a shared secret (session key).
5	Encrypted Session	A secure (encrypted) connection is established and HTTP data is sent over it.

Observable Metadata (Without Decryption)

You Can See	Example
Source and destination IPs	192.168.1.10 → 93.184.216.34
Ports	Client: 54321, Server: 443
Protocol	TLS (over TCP)
TLS version	TLS 1.3
Server name (SNI)	example.com
Certificate details	Domain, issuer, and validity period

Diagnostic Command

```
openssl s_client -connect example.com:443 -servername example.com
```

- Connects to the HTTPS service on port 443.
- Requests the certificate for example.com.
- Displays the TLS version, cipher, and certificate details.

Example Output (Key Lines)

```
...
CONNECTED(0000003)
Protocol    : TLSv1.3
Cipher      : TLS_AES_256_GCM_SHA384
Server certificate
subject=CN = example.com
verify return:1 (ok)
...
```

Security Insight

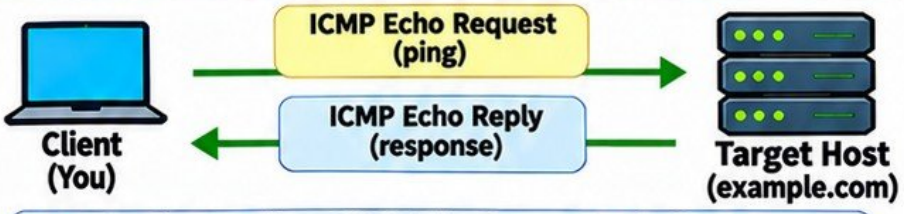
Never assume encrypted traffic is suspicious simply because its contents are hidden.

ICMP: What Ping Really Tells You

Key Takeaways

- Understand ICMP Echo Request and Echo Reply.
- Interpret round-trip time (RTT) and packet loss.
- Distinguish an unreachable host from a host that blocks ICMP.
- Use ping -c 4 and traceroute to investigate connectivity.
- Know that a failed ping does not always mean the application is down.

How ICMP Ping Works



- The client sends an ICMP Echo Request to the host.
- The host replies with an ICMP Echo Reply.
- The time between the request and reply is the round-trip time (RTT), measured in milliseconds (ms).

Interpreting Ping Results

Result	Meaning
Replies with RTT (e.g., 24 ms)	The host is reachable, and ICMP is allowed.
Packet loss (e.g., 25%)	Some requests did not get a reply (network congestion, link issues, or rate limiting).
Destination Unreachable	The network cannot reach the host (e.g., no route or the host is down).
No Reply (timeout)	The host may be up but is blocking ICMP (e.g., by a firewall).

Diagnostic Commands

```
>_ ping -c 4 example.com
>_ traceroute example.com
```

- ping -c 4: Sends 4 ICMP echo requests and shows RTT and packet loss.
- traceroute: Shows the path (hops) to the destination and where delays occur (if installed).

Example: Ping Output

```
$ ping -c 4 example.com
PING example.com (93.184.216.34) 56(84) bytes of data.
64 bytes from 93.184.216.34: icmp_seq=1 time=24.7 ms
64 bytes from 93.184.216.34: icmp_seq=2 time=24.3 ms
64 bytes from 93.184.216.34: icmp_seq=3 time=24.6 ms
64 bytes from 93.184.216.34: icmp_seq=4 time=24.5 ms
--- example.com ping statistics ---
4 packets transmitted, 4 received, 0% packet loss,
round-trip min/avg/max = 24.3/24.5/24.7 ms
```

Useful Comparison

Scenario	What It Suggests
Ping succeeds, low RTT	Host is reachable and responsive.
Ping fails (timeout)	Host may be up but blocking ICMP.
Destination unreachable	No route to the host (network issue).
High RTT or packet loss	Network congestion or a performance issue.

⚠ Beginner Mistake

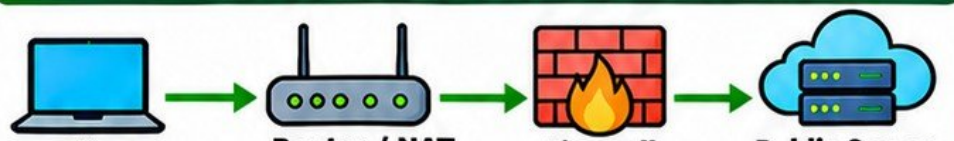
✗ **Assuming a failed ping proves the application is unavailable.**

Follow a Request Through Routers and Firewalls

Key Takeaways

- Understand gateways, routing, hops, and packet forwarding.
- Explain how firewall rules affect traffic (allow or block).
- Introduce Network Address Translation (NAT).
- Follow a private client IP communicating with a public server.
- Use ip route and ip route get 1.1.1.1 to inspect routing.
- Investigate a cloud server that cannot reach an external API.

How the Traffic Flows (Example)



Client (Private IP) 192.168.1.10 → Router / NAT (Gateway) 192.168.1.1 → 203.0.113.5 → Firewall (Rules: Allow / Block) → Public Server (External API) api.example.com (198.51.100.10)

- The client (private IP) sends traffic to the default gateway (router).
- The router forwards the traffic and uses NAT to replace the private IP with a public IP (203.0.113.5).
- The firewall checks the traffic against its rules (allow or block).
- The request then reaches the external server (e.g., an API).

Important Concepts

Concept	Meaning
Gateway	The next hop that sends traffic outside your network.
Routing	The process of choosing the path for traffic to reach a destination.
Hop	A device (e.g., router) the traffic passes through.
Firewall	A security control that allows or blocks traffic based on rules.
NAT	Translates a private IP (e.g., 192.168.x.x) to a public IP for internet access.

Diagnostic Commands

```
>_ ip route
>_ ip route get 1.1.1.1
```

- ip route: Shows the routing table (known networks and next hops).
- ip route get 1.1.1.1: Shows the exact path and the source IP that will be used to reach 1.1.1.1 (e.g., the gateway and interface).

Sample Outputs

```
$ ip route
default via 192.168.1.1 dev eth0
192.168.1.0/24 dev eth0 proto kernel scope link
```

```
$ ip route get 1.1.1.1
1.1.1.1 via 192.168.1.1 dev eth0
src 192.168.1.10
```

Production Connection



Scenario: A cloud server cannot reach an external API.

Investigate:

1. Check the routing path (ip route).
2. Confirm the egress IP and NAT behavior.
3. Verify firewall/security group rules (allow outbound traffic on the required port).
4. Test connectivity to the API (e.g., ping, curl).

Key Insight

A connection can fail anywhere along the path — routing, NAT, or firewall — not just at the server.

Wireshark Filters That Reveal the Important Packets

Key Takeaways

- Distinguish capture filters from display filters.
- Use common filters: dns, tcp, tcp.port == 443, and ip.addr == 192.0.2.10.
- Identify retransmissions with tcp.analysis.retransmission.
- Follow a TCP stream to reconstruct the visible conversation.
- Isolate packets for one failed request.
- Remember: display filters hide packets from view; they do not remove them from the capture.

Capture Filter vs Display Filter

Capture Filter (e.g., tcp port 443)	Display Filter (e.g., dns)
Applied before capture (stops unwanted packets from being saved).	Applied after capture (only hides what you don't want to see).
Reduces capture size and storage.	The packets are still in the capture file.

Useful Wireshark Display Filters

Filter	What It Shows
dns	DNS queries and responses (name lookups).
tcp	All TCP traffic (connections, data, and control packets).
tcp.port == 443	TCP traffic to port 443 (HTTPS).
ip.addr == 192.0.2.10	All traffic to or from the IP 192.0.2.10.
tcp.analysis.retransmission	TCP packets that were retransmitted (often a sign of packet loss or delay).

Follow a TCP Stream



Wireshark (TCP Stream)

1. Apply a filter (e.g., tcp.port == 443).
2. Right-click a packet in the conversation.
3. Select Follow → TCP Stream.
4. View the reconstructed request and response (visible for unencrypted traffic).

Example: Failed Request Investigation

`tcp.stream == 5`

1. Filter to the failed connection (e.g., tcp.stream == 5).
2. Inspect the flow (SYN, SYN-ACK, ACK, request, response).
3. Check for issues (retransmissions, reset, or missing response).

Important Warning

Display filters hide packets from view; they do not remove them from the capture.

TCP Flags: Read the Connection's Behavior

Key Takeaways

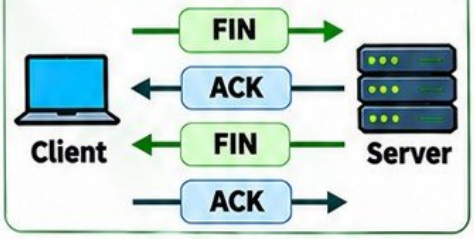
- Understand TCP flags: SYN, ACK, FIN, and RST.
- Distinguish normal connection closure from an abrupt reset.
- Explain how sequence and acknowledgment numbers help track delivery.
- Identify duplicate ACKs and zero-window advertisements.
- Know that a reset reveals connection termination, but further evidence is needed to explain the cause.

Common TCP Flags and Meaning

Flag	Meaning
SYN	Start a connection (synchronize sequence numbers).
ACK	Acknowledge received data (confirms delivery).
FIN	Gracefully close a connection (no more data to send).
RST	Immediately terminate a connection (abort/reset).

Connection Flows: Normal vs Abrupt

Normal Connection Closure (Graceful)



Abrupt Reset (Immediate Termination)



Sequence and Acknowledgment Numbers

Term	Meaning
Sequence Number (Seq)	The number of the first byte in this segment.
Acknowledgment Number (Ack)	The next byte the sender expects to receive.

Other Important Signals

Signal	Meaning
Duplicate ACK	The same ACK is received multiple times, often indicating a lost packet.
Zero-Window Advertisement	The receiver reports a window size of 0, meaning it cannot accept more data right now (temporary pause/backpressure).

Example: TCP Flags in a Session

No.	Flag	Seq	Ack	Info
1	SYN	0	-	Client → Server (start)
2	SYN,ACK	0	1	Server → Client (agree)
3	ACK	1	1	Client → Server (established)
...	(data)	...	...	Data transfer ...
10	FIN	1024	2048	Client → Server (close)
11	ACK	2048	1025	Server → Client

Detective Insight

A reset reveals a connection termination, but further evidence is needed to establish why it happened.

Slow Network or Slow Application?

Key Takeaways

- Distinguish DNS lookup time, TCP connection time, TLS negotiation time, and server response delay.
- Use curl -w timing metrics to investigate requests.
- Explain time to first byte and its limitations.
- Compare two request timelines.
- Decide whether to investigate networking, server processing, or a downstream dependency next.

Where Time Is Spent



DNS Lookup Time (name resolution)	TCP Connection Time (3-way handshake)	TLS Negotiation Time (encryption setup)	Server Response Delay (application processing)

Use curl -w Timing Metrics

```
curl -w "\
     DNS:   %{time_namelookup}s
     TCP:   %{time_connect}s
     TLS:   %{time_appconnect}s
     Start: %{time_starttransfer}s
     Total: %{time_total}s
   -o /dev/null https://example.com
```

Key Metrics

time_namelookup	– DNS time
time_connect	– TCP time
time_appconnect	– TLS time
time_starttransfer	– Time to first byte
time_total	– Total time

What Is Time to First Byte?

Time to first byte (TTFB) is the time from the request until the first byte of the response is received.

Limitations

- Does not include the full response time.
- Can be affected by caching, CDNs, and intermediaries.
- Does not show what happens after the first byte is sent.

Compare Two Request Timelines

Stage	Request A (Fast)	Request B (Slow)
DNS Lookup	40 ms	45 ms
TCP Connect	80 ms	120 ms
TLS Handshake	90 ms	150 ms
TTFB	200 ms	1,800 ms
Total Time	1,100 ms	3,200 ms

Request B is slower at TTFB and server response, suggesting a server or application issue (not just network).

Network or Application?

- High DNS time → Check DNS / resolver.
- High TCP time → Check routing, firewall, network.
- High TLS time → Check certificates, SSL/TLS setup.
- High TTFB / total time → Check server, app, or downstream dependency.

Practice

Decide whether to investigate networking, server processing, or a downstream dependency next.

Remember

Slow does not always mean the network. The problem could be deeper.

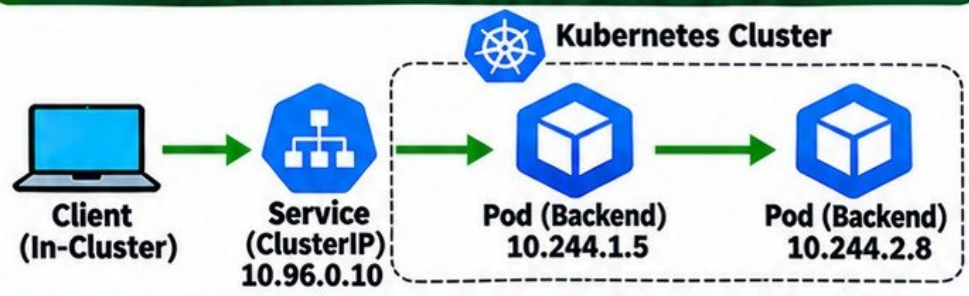
VERIQTA

When Containers and Kubernetes Hide the Clues

Key Takeaways

- Understand container network namespaces, Pod IPs, Services, and traffic forwarding.
- Explain why localhost inside a Pod is not the host machine.
- Diagnose a Kubernetes Service that cannot reach its backend.
- Use kubectl get pods -o wide, kubectl get svc, and kubectl get endpointslices.
- Explain why traffic captured at different points may show different IP addresses.
- Remember: capture location matters as much as the packets themselves.

How Traffic Flows in Kubernetes



Important Concepts

Concept	Meaning
Network Namespace	A separate network view for each container (isolated networking).
Pod IP	The IP address assigned to a Pod (e.g., 10.244.x.x).
Service	A stable virtual IP and DNS name that forwards traffic to one or more Pods.
Traffic Forwarding	Routing from client → Service → Pod (backend).

Key Commands

`>_ kubectl get pods -o wide`
- List Pods with their IP addresses and node details.

`>_ kubectl get svc`
- List Services, including ClusterIP and ports.

`>_ kubectl get endpointslices`
- Show the backend Pod IPs (endpoints) that a Service forwards traffic to.

localhost in a Pod



localhost (127.0.0.1) inside the Pod refers to the Pod itself, not the Kubernetes node or the host machine.

Why IPs May Look Different

- Inside a Pod: you see the Pod IP (e.g., 10.244.x.x).
- At the Service: you see the Service ClusterIP (e.g., 10.96.x.x).
- At the node or external view: you may see the node IP or a NATed IP (e.g., 192.168.x.x).

Diagnose: Service Cannot Reach Backend

1. Check Pods: kubectl get pods -o wide (are the backend Pods running and Ready?).
2. Check Service: kubectl get svc (is the Service created with the correct port?).
3. Check Endpoints: kubectl get endpointslices (does it list the expected Pod IPs?).
4. Test from a Pod: curl http://<service-name>:<port> (verify if the Service can reach the backend).

⚠ Production Insight



Capture location matters as much as the packets themselves.

Recognize Suspicious Traffic Without Jumping to Conclusions

Key Takeaways

- Recognize repeated connection attempts, unusual destination ports, and unexpected outbound connections.
- Distinguish legitimate scanning, monitoring, retries, and potentially malicious behavior.
- Introduce connection baselines and traffic volume.
- Explain why packet patterns alone cannot prove an attack.
- Investigate an unfamiliar outbound connection using approved logs and system information.

Common Suspicious Traffic Indicators



Repeated Attempts

Many connection tries to the same IP/port.



Unusual Ports

Traffic to ports not normally used (e.g., 4444, 8081).



Unexpected Outbound

A host connects to an unknown or unapproved external IP.

Legitimate vs Potentially Malicious Traffic

Behavior	Possible Legitimate Reason	Potentially Malicious If ...
Scanning	Asset discovery, vulnerability assessment, or monitoring.	From an unknown source, repeated and targeted.
Retries	Normal application retry after a temporary failure.	Excessive, frequent, and over a long period.
Outbound Connection	Software update, license check, telemetry, or API call.	To an unrecognized or high-risk destination with unusual ports.

Build a Traffic Baseline



- Learn what normal traffic looks like (typical destinations, ports, and volume).
- Identify changes or spikes over time.
- Use baselines to spot what is unusual for your environment.

Important Reminder



Packet patterns alone cannot prove an attack. They are a signal to investigate, not a final conclusion.

Investigate an Unfamiliar Outbound Connection

1.  Identify the connection (e.g., ss -tunap or netstat -tunap).
2.  Check system and application logs (e.g., journalctl, /var/log/).
3.  Find the process and user that initiated it (e.g., ps -fp <PID>).
4.  Assess the destination (reputation, ownership, expected business use).
5.  Take action based on evidence (allow, block, or escalate).

Summary



- Look for patterns and context.
- Compare with your baseline.
- Use logs and system info to confirm the cause.

Build a Network Incident Timeline

Key Takeaways

- Correlate packet timestamps with application logs and infrastructure events.
- Record the first failure, affected destination, retries, resets, and recovery.
- Understand timestamp accuracy, clock synchronization, and time zones.
- Distinguish observations, hypotheses, and confirmed findings.
- Create a clear, chronological timeline of the incident.

Correlate Evidence from Multiple Sources



Packet Capture (network events with timestamps)

+



Application Logs (application errors and events)

+



Infrastructure Events (services, scaling, restarts, deployments)

Timeline of a Network Incident (Example)

Time (UTC)	Event	Evidence (Source)
10:05:12	First failure to reach API (HTTP 502).	Application log + packet (502)
10:05:45	Automatic retries begin (repeated requests).	Packet capture (multiple attempts)
10:06:30	Connection reset by server (TCP RST).	Packet capture (RST flag)
10:07:05	Service recovers (HTTP 200).	Application log + packet (200)
10:07:30	Traffic returns to normal.	Metrics / infrastructure logs (stable).

Time, Clocks and Time Zones



- Timestamp accuracy: how precise the time is on each system.
- Clock synchronization: keep systems in sync (e.g., NTP).
- Time zones: convert to a single reference (e.g., UTC) for analysis.

Observation, Hypothesis, and Finding

Type	Meaning
Observation	A factual event from the evidence (e.g., retries at 10:05:45).
Hypothesis	A possible explanation to investigate (e.g., upstream service was overloaded).
Confirmed Finding	A conclusion supported by evidence (e.g., upstream service was overloaded from 10:05 to 10:07).

Practice: Create a Five-Event Timeline



1. The first failure (time and destination).
2. The retry behavior (time and pattern).
3. The reset or error response (e.g., TCP RST or 5xx).
4. The recovery event (time and success status).
5. Confirmation of stability (normal traffic restored).

Important Insight



A clear, time-based timeline turns scattered events into a coherent incident story.

Case Study: The Website That Randomly Times Out

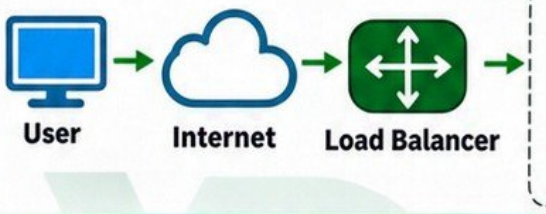
Scenario

- Users report intermittent timeouts behind a load balancer.

Evidence

- DNS resolves, some TCP connections succeed, and others receive no response.

Network Flow Overview



Compare Connection Attempts

Attempt	Result	Details
1	Success	TCP handshake completes, HTTP 200.
2	Success	TCP handshake completes, HTTP 200.
3	Timeout	No response after SYN (retries).
4	Timeout	No response after SYN (retries).

Key Investigation Areas

Load Balancer
- Target health
- Connection limits

Security Rules
- Firewall / SG rules
- ACLs / NACLs

Backend Health
- Instance status
- Application logs

Configuration
- Timeouts
- Routing rules

Hypothesis vs Evidence

Hypothesis	Supporting Evidence	Rejecting Evidence
Load balancer misconfiguration	Timeouts only on some requests; others succeed.	All targets healthy and reachable.
Security group blocking traffic	No response (SYN retries) to specific targets.	Rules allow traffic for other targets.
Backend server unhealthy	One or more targets unresponsive.	All targets respond correctly.
Application slowness	Long response times before timeout.	Fast responses from healthy targets.

Verify Before You Fix

1. Confirm the actual cause with logs, metrics and tests.
2. Reproduce the issue when possible.
3. Validate the fix in staging or with a small change.
4. Monitor after the fix to ensure stability.

Production Lesson

A network symptom can originate in infrastructure, configuration, or application behavior.

Remember

Don't assume the problem is the application. Follow the evidence.

Final Investigation: Solve the Incident From Packet Evidence

Incident Scenario

The payments API at api.veriqta.com is unreachable for many users. The web app returns timeouts when calling the API.

Learner Task

1. Identify the first failed stage of communication.
2. Determine which evidence is confirmed and which remains uncertain.
3. Select the next diagnostic command and explain why.
4. Recommend a safe corrective action and verification procedure.
5. Produce a short incident report (symptoms, evidence, root cause/hypotheses, remediation, prevention).

Investigation Evidence (Provided)

- Packet Timeline (key events)
- DNS Results
- HTTP Observations (curl)
- Application Logs

Key Evidence

Packet Timeline (Client → API)

Time (UTC)	Event
10:24:10.100	DNS query for api.veriqta.com
10:24:10.120	DNS response: 203.0.113.25
10:24:10.260	TCP SYN (to 203.0.113.25:443)
10:24:13.260	No SYN-ACK (retries)
10:24:19.260	Connection timeout

DNS Results

```
$ dig api.veriqta.com
api.veriqta.com.        300 IN A 203.0.113.25
```

HTTP Observation

```
$ curl -I https://api.veriqta.com
curl: (28) Connection timed out after 10000 ms
```

Application Log (API Client)

```
2026-11-03 10:24:10 INFO  Calling https://api.veriqta.com/pay
2026-11-03 10:24:20 ERROR Request timed out after 10s (no response)
```

What It Shows

- DNS works (name resolves).
- TCP connection is attempted but not completed.
- No HTTP response (connection fails before the application can send a request).

Recommended Next Step

```
traceroute api.veriqta.com
```

- Confirms the exact hop where the connection stops.
- Helps determine if the issue is a network path, firewall/security rule, or a target (endpoint) problem.

Corrective Action & Verification

- Corrective action: Allow HTTPS (port 443) to 203.0.113.25 on the firewall / security group or fix the target's listener.
- Verification:
  - Retest with: curl -I https://api.veriqta.com
  - Confirm successful TCP handshake.
  - Check application logs for successful calls.

Short Incident Report (Example)

Symptoms	Users experienced timeouts when the web app called api.veriqta.com.
Evidence	DNS resolved to 203.0.113.25, TCP SYN sent, no SYN-ACK, connection timed out, application logs show 10s timeout.
Root Cause / Hypotheses	Most probable: HTTPS traffic to the API was blocked by a firewall/security rule (or the target listener was not reachable).
Remediation	Allow HTTPS (port 443) to the API target and ensure the target listener is healthy.
Prevention	Maintain change records for firewall rules, add connectivity monitoring, and set alerts for API reachabilty and timeouts.